Metame SDK – Unity Package Guide

This guide explains how to set up a character and import the Metame SDK into your Unity project. The SDK is provided as separate `.unitypackage` files for each render pipeline: **Built-in**, **URP (Universal Render Pipeline)**, and **HDRP (High Definition Render Pipeline)**.

Requirements

- Unity Version: 2022+ LTS or newer (Recommended: Unity 6)
- Scripting Backend: IL2CPP or Mono
- Supported Render Pipelines: Built-in, URP, HDRP

1. Choose the Correct SDK Package

Metame is distributed as three different `.unitypackage` files. Select and import only the one that matches the render pipeline used in your Unity project.

Package Options

Render Pipeline	Package Name	Description
Built-in	Metame-BuiltInRP.unitypackage	For projects using the default Unity pipeline
URP	Metame-URP.unitypackage	For projects using Universal Render Pipeline
HDRP	Metame-HDRP.unitypackage	For projects using High Definition Render Pipeline

2. Download the Correct Package

Download the appropriate package for your render pipeline from the Metame SDK download page.

Example download links:

- Metame-BuiltInRP.unitypackage //Link
- Metame-URP.unitypackage //Link
- Metame-HDRP.unitypackage //Link

Ensure that you only import one of the three packages into your project.

3. Import the Package into Unity

Follow these steps to import the selected `.unitypackage`:

1. Open your Unity project.
2. In the top menu, go to `Assets > Import Package > Custom Package...`.
3. Select the `.unitypackage` file you downloaded.
4. In the import dialog, confirm that all items are selected, then click `Import`.

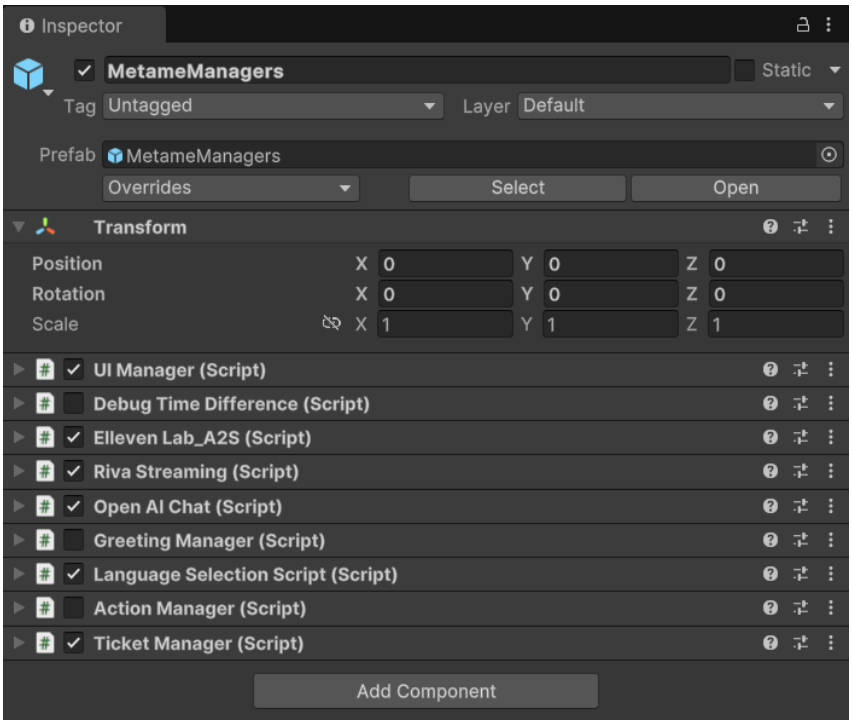
4. Testing the Metame SDK

Once you've imported the Metame SDK into your project, it's a good idea to test the setup by opening the provided demo scene. Follow these steps to check if the SDK is functioning correctly:

Steps to Open the Demo Scene:

1. **Locate the Demo Scene:**
    - In the **Project** window, go to `Assets > ProjectData > Scenes`.
    - Double-click on `MetameDemo` to open the demo scene.
2. **Verify the Scene Setup:**
    - After the scene loads, go to the **Hierarchy** window.
    - You should see a `MetameManager` object in the hierarchy.

3. **Check the MetameManager:**
    - Select the `MetameManager` object in the **Hierarchy** window.
    - In the **Inspector**, you will see all the scripts and components that are responsible for making Metame work within your project.



This will allow you to verify that the SDK is functioning as expected and that all necessary scripts are correctly attached to the `MetameManager`.

5. Importing and Setting Up a Character

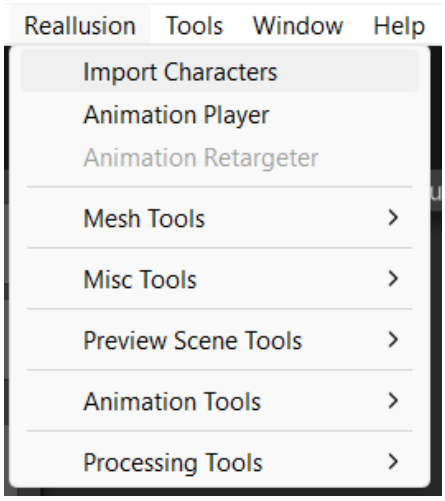
Currently, the Metame SDK supports only characters exported from **Character Creator 4 (CC4)**. Follow the steps below to properly import and configure a CC4 character in your Unity project.

Step 1: Import Your CC4 Character

1. Export your character from **Character Creator 4**.
2. In Unity, **drag and drop** the exported character folder into your project's **Project window**.

Step 2: Use the Reallusion Import Tool

Once your character is added to the project, go to the Unity top menu bar and click on: `Reallusion > Import Characters`



1. A new window will appear showing the list of available characters.
2. Select your character name from the list.
3. Click **Build Materials**. This will generate materials and finalize the import.

Step 3: Add the Character to the Scene

1. In the **Project window**, locate the prefab of your imported character.
2. Drag and drop the prefab into the **Scene view** or **Hierarchy** window.

6. Configuring the ConversationManager Component

After placing your character into the scene, follow these steps to enable Metame's conversational features:

1. **Select the character** in the **Hierarchy**.
2. In the **Inspector** window, click **Add Component**.

Add a script/component called:

`ConversationManager`

ConversationManager Fields

Fill out the required fields in the ConversationManager:

- **Voice**
  Select any voice available in your project for the character's spoken output.

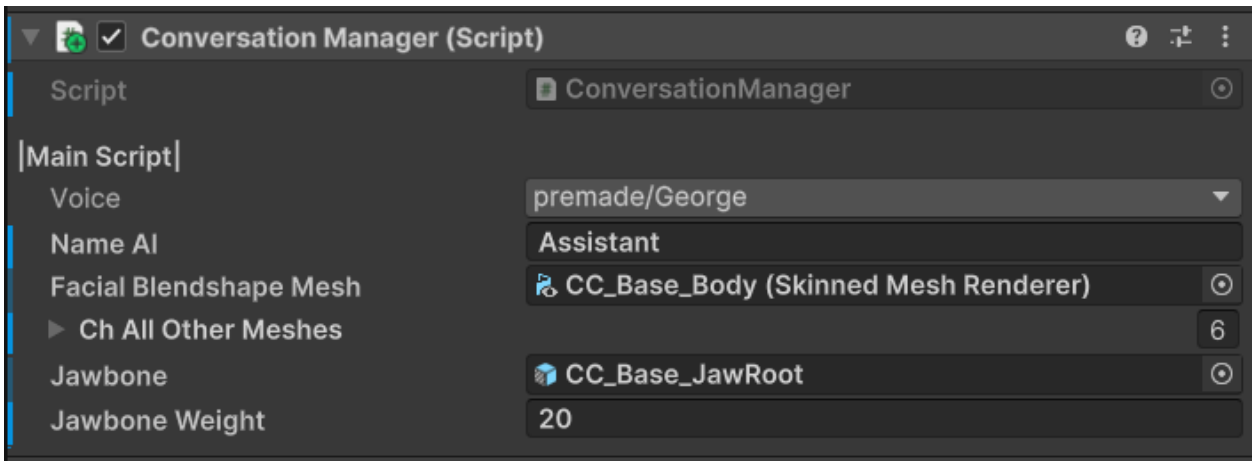
- **Name AI**
  Assign a name for your AI character. This will be used in conversations and UI references.

- **Facial Blendshape Mesh**
  Drag and drop the **Skinned Mesh Renderer** responsible for facial expressions. This should be the main face mesh with blendshapes.

- **Ch All Other Meshes**
  Optional. Assign any additional Skinned Mesh Renderers (such as separate eyebrow or eyelash meshes) that also have blendshapes with the same names as the main mesh.

- **Jawbone**
  Drag and drop the **Jaw Root bone** from the character's rig.

- **Jawbone Weight**
  Integer value. This acts as a multiplier to control how much the jaw opens during speech. Adjust based on the character's anatomy and rig.



7. Auto-Add Required Components

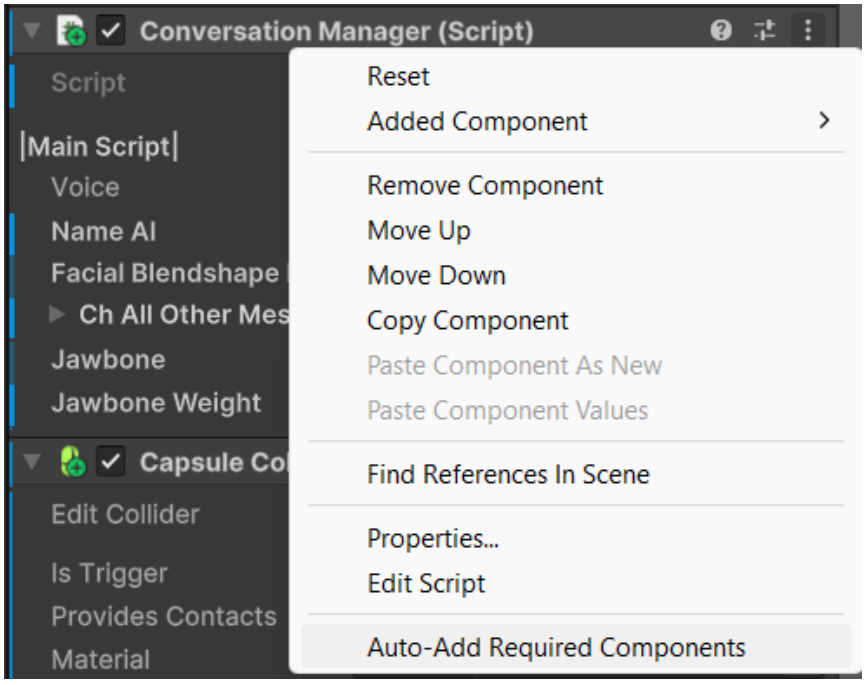
Once the `ConversationManager` component has been added and configured, Metame allows you to automatically attach additional required components that enable full functionality such as head tracking, blinking, and audio playback.

Step-by-Step Instructions:

1. In the **Inspector** window, locate the `ConversationManager` component on your character.

2. Click the **three dots (⋮)** icon at the top-right corner of the `ConversationManager` component.

From the dropdown menu, select:
`Auto-Add Required Components`



This action will automatically add the following components to your character:

Added Components:

- **Head Tracking Solution**
  Enables the character to rotate its head and look at a designated target in the scene.

This is used to simulate natural head movement during interaction.

- **Blink Controller**
  Handles eye blinking animation using facial blendshapes.
  Helps bring life and realism to the character.

- **Audio Source**
  Unity's built-in `AudioSource` component used to play voice responses.
  After it is added, **manually adjust the `Spatial Blend` setting**:

  - Select the Audio Source component.

  - Set `Spatial Blend` to **3D** to make voice playback positional in 3D space.

8. Configuring the Head Tracking Solution

The **Head Tracking Solution** component allows your character to dynamically rotate its head, eyes, and optionally its body to look at a target GameObject (such as the player or a camera). This enhances realism and interaction during conversations.

Component Fields and Their Functions:

- **Target Object**
  Assign the **GameObject** that the character should look at.
  Example: The player character or the main camera.

- **Tracking Distance**
  A float value that determines **how close** the target must be before the character begins tracking it.
  The character will only engage in looking behavior when the target is within this range.

- **Turn Speed**
  Controls how quickly the **body rotates** to face the target.
  A higher value will result in faster turning behavior.

- **Body Look At Weight**
  A value between **0 and 1**.
  Determines how much the **full body** contributes to the look-at motion.

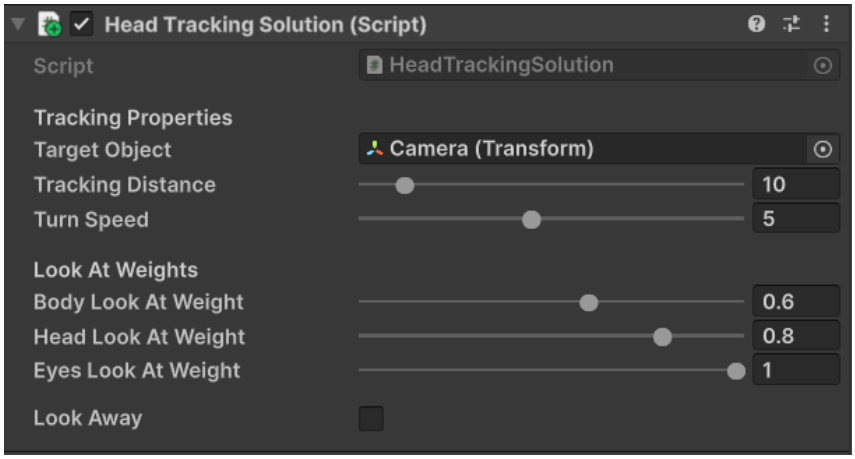
  - `0`: No body movement

    - `1`: Full body will turn

- **Head Look At Weight**
  A value between **0 and 1**.
  Determines how much the **head rotation** contributes to tracking.
    - `0`: No head turning
    - `1`: Full head turning toward the target

- **Eye Look At Weight**
  A value between **0 and 1**.
  Controls the intensity of **eye movement** toward the target.
    - `0`: Eyes stay static
    - `1`: Full eye tracking

- **Look Away**
  A boolean toggle (`true` / `false`).
  When enabled, the character will occasionally **look away** from the target dynamically to mimic natural behavior.



This component works in real time to give characters the ability to track and interact with players or objects naturally. Be sure to test different weight values for the most realistic results based on your character's rig and animation style.

9. Configuring the Blink Controller

The **Blink Controller** component manages eye blinking using facial blendshapes. It helps make the character feel more lifelike during conversations and idle states.

Component Fields and Descriptions:

- **Eye Skinned Mesh Renderer**
  Drag and drop the **Skinned Mesh Renderer** that contains the eye-related blendshapes.
  This is usually the same mesh used for facial expressions.

- **Index Of Left Eyelid**
  Automatically detected by the script.
  You can leave this field empty unless you want to manually override the blendshape index for the left eyelid.

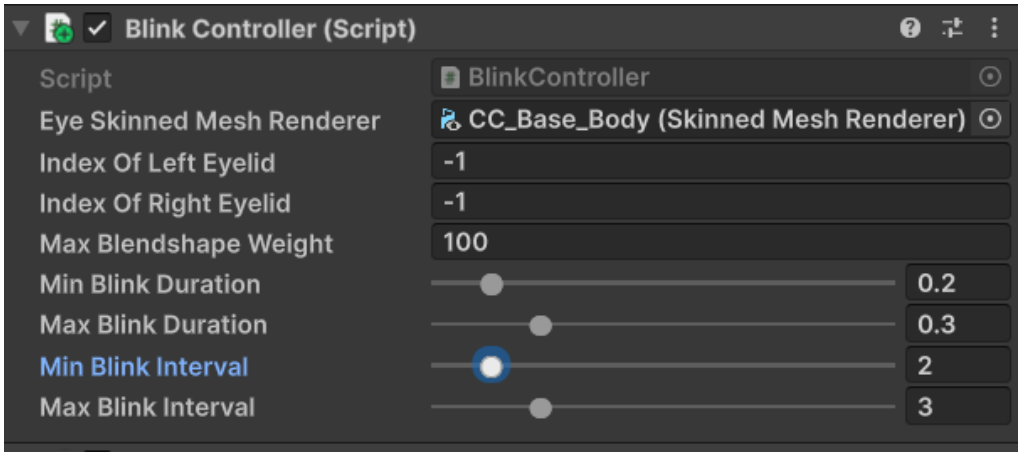
- **Index Of Right Eyelid**
  Also auto-detected.
  No manual input is required unless needed for advanced customization.

- **Min Blink Duration**
  Float value between **0 and 1**.
  Determines the **shortest duration** (in seconds) of a single blink.

- **Max Blink Duration**
  Float value between **0 and 1**.
  Controls the **longest possible duration** for a blink.

- **Min Blink Interval**
  Float value between **0 and 10**.
  Sets the **minimum time interval** (in seconds) between blinks.

- **Max Blink Interval**
  Float value between **0 and 10**.
  Sets the **maximum time interval** (in seconds) between blinks.



Notes:

- The script will randomly choose blink durations and intervals between the min and max values.
- You should test different ranges to match the natural blinking behavior of your character's facial rig.
- Blinking will continue to run even if the character is idle or not talking, for realism.

Important Notes

- Only one Metame package should be imported per project. Do not import more than one pipeline version.
- If you need to switch pipelines, fully remove the previously imported Metame package before importing a new one.
- Verify that your project is correctly set up with the appropriate Render Pipeline Asset under `Edit > Project Settings > Graphics`.

Troubleshooting

Issue	Solution
Materials appear pink	This usually means the wrong pipeline package was imported.
Shader errors after import	Make sure you're using the correct version of Unity and render pipeline.
Duplicate assets or scripts	Ensure only one Metame package is present in the project.